SI-VISION ACADEMY

Design and Implementation of CSI Controller IP

CSI-2 Transmitter

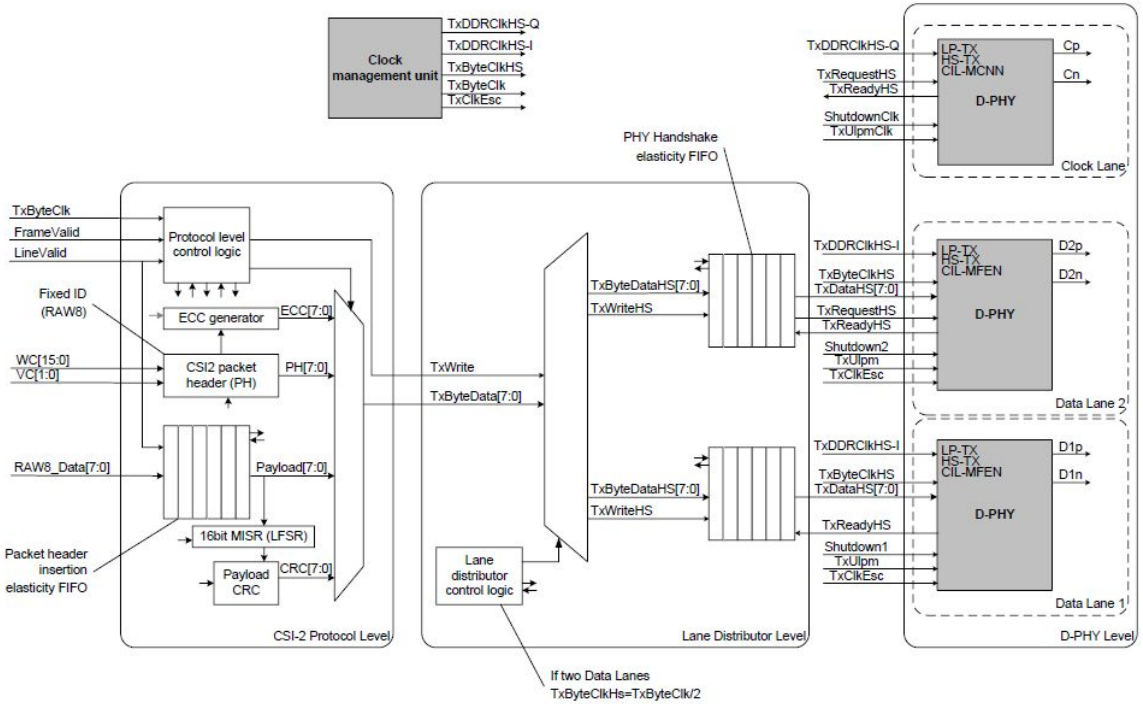
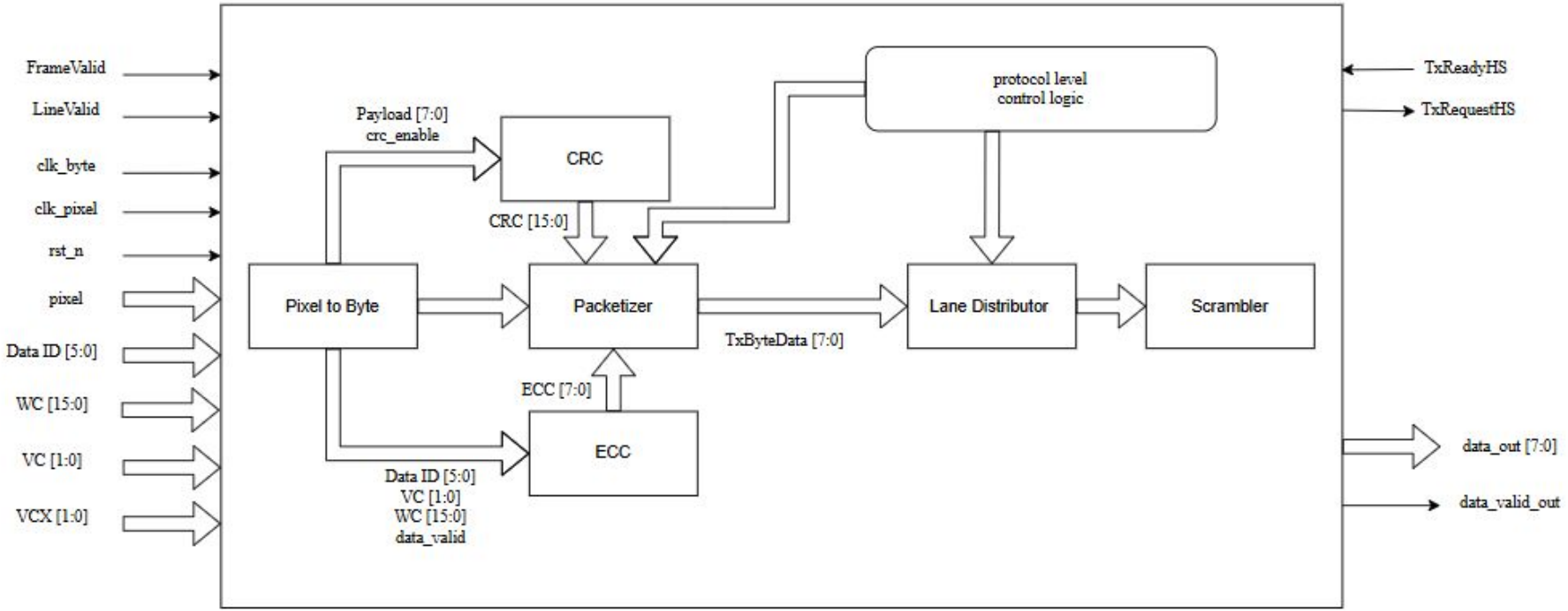


Figure 213 CSI-2 Transmitter Block Diagram

CSI-2 Transmitter



SI-VISION ACADEMY

Number of lanes

Number of lanes

CSI Signaling Mode

This MIPI CCS Specification supports two main signaling modes for the image data interface:

- CSI-2 using D-PHY
- CSI-2 using C-PHY

The Host can determine which by reading the default value of the register ***CSI_signaling_mode***. Mode switching from D-PHY to C-PHY

Register Name	Type	RW	Comment
CSI_signaling_mode	8-bit unsigned integer	RW	0: Reserved 1: Reserved 2: CSI-2 with D-PHY 3: CSI-2 with C-PHY **Default:** Image–sensor-specific (2 or 3)

Number of lanes

CSI Lane Mode

The ***CSI_lane_mode*** register controls the Lane configuration used with either D-PHY or with C-PHY. The Host shall configure the number of used Lanes.

Register Name	Type	RW	Comment
CSI_lane_mode	8-bit unsigned integer	RW	0: 1-Lane 1: 2-Lane 2: 3-Lane 3: 4-Lane 4: 5-Lane 5: 6-Lane 6: 7-Lane 7: 8-Lane **Default:** Image-sensor-specific (0 to 7).

SI-VISION ACADEMY

Transmitter

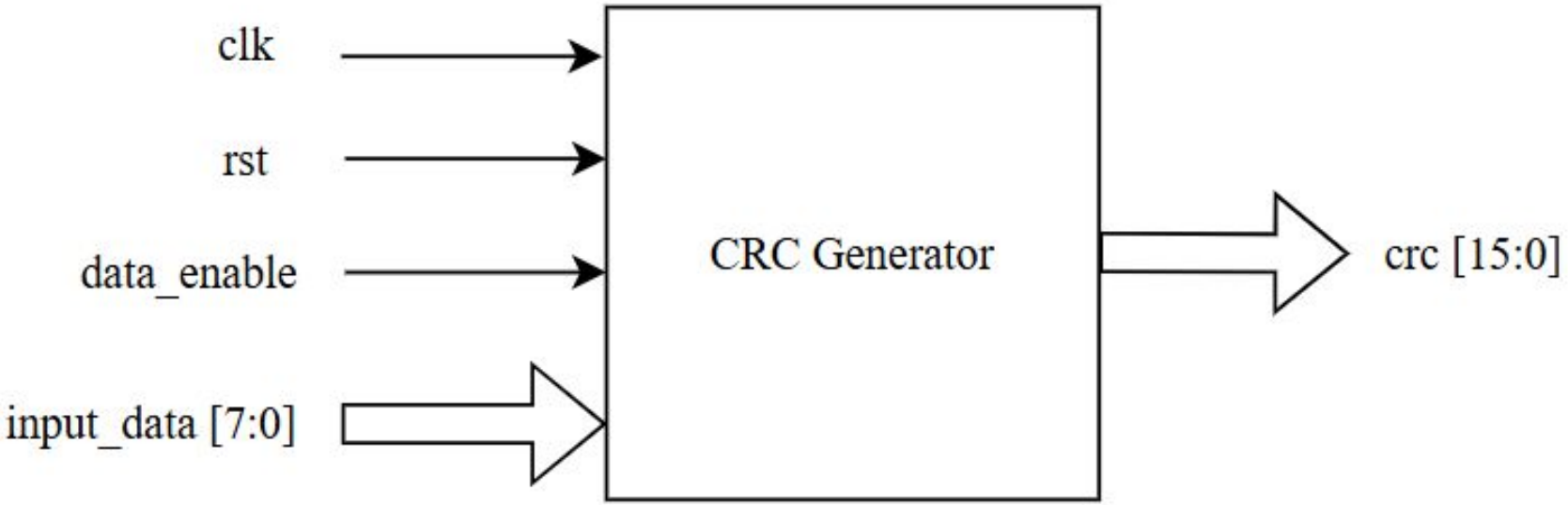
SI-VISION ACADEMY

CRC

Cyclic Redundancy Check

At transmitter side:

CRC Calculation



Cyclic Redundancy Check

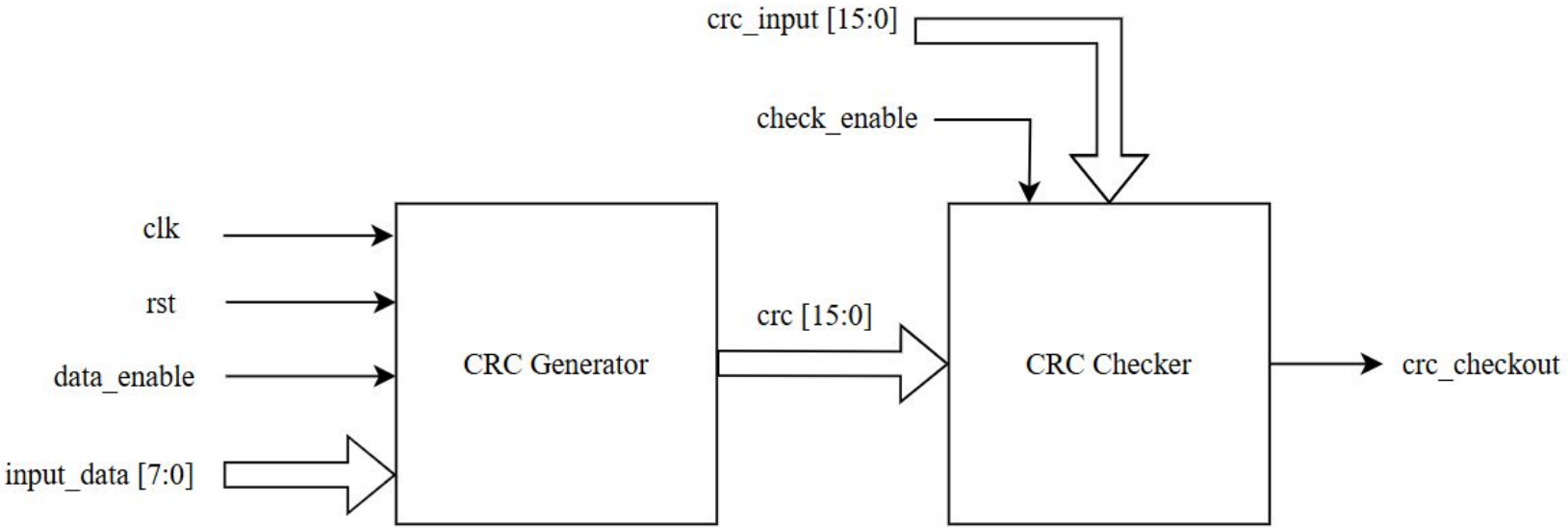
At transmitter side:

Signal Name	Direction	Description
CRC Generator		
clk	Input	Clock signal
rst	Input	Asynchronous reset signal
data_enable	Input	Input data enable signal
input_data [7:0]	Input	Input data being transmitted
crc [15:0]	Output	CRC code

Cyclic Redundancy Check

At receiver side:

CRC Calculation then error detect



Cyclic Redundancy Check

At receiver side:

Signal Name	Direction	Description
CRC Checker		
clk	Input	Clock signal
rst	Input	Asynchronous reset signal
check_enable	Input	Input enable signal
crc_input [15:0]	Input	Received CRC code
crc_checkout [15:0]	Output	CRC checker out

SI-VISION ACADEMY

Error Correction Code

Hamming Code Example

Input Data: 8 bits (D7 D6 D5 D4 D3 D2 D1 D0)

- Example Value: `00111001`

Check Bits Required: For 8 data bits, we can use this form to know **`2^r≥n+r+1`**, where n = number of bits, and r = number of ECC (parity) bits for SEC,then add +1 bit for SECDED, Hamming code requires **4 check bits** (often placed at positions C1, C2, C4, C8) to enable single-bit error correction.

Total Bits Transmitted:

- Data Bits: 8
- Check Bits: + 4
- **Total: 12 bits** (Codeword)

Hamming Code

- Example Value: `00111001`

	D7	D6	D5	D4	C8	D3	D2	D1	C4	D0	C2	C1	
	12	11	10	9	8	7	6	5	4	3	2	1	
	0	0	1	1	C8	1	0	0	C4	1	C2	C1	**C0**
C1	0	1	0	1	0	1	0	1	0	1	0	1	**0**
C2	0	1	1	0	0	1	1	0	0	1	1	0	**0**
C4	1	0	0	0	0	1	1	1	1	0	0	0	**0**
C8	1	1	1	1	1	0	0	0	0	0	0	0	**0**

Hamming Code

	D7	D6	D5	D4	C8	D3	D2	D1	C4	D0	C2	C1	
	12	11	10	9	8	7	6	5	4	3	2	1	
	0	0	1	1	C8	1	0	0	C4	1	C2	C1	C0
C1	0	1	0	1	0	1	0	1	0	1	0	1	0

Calculation of C1:

C1 = D0 ⊕ D1 ⊕ D3 ⊕ D4 ⊕ D6 =

C1 = 1 ⊕ 0 ⊕ 1 ⊕ 1 ⊕ 0 = 1

Hamming Code

	D7	D6	D5	D4	C8	D3	D2	D1	C4	D0	C2	C1	
	12	11	10	9	8	7	6	5	4	3	2	1	
	0	0	1	1	C8	1	0	0	C4	1	C2	C1	**C0**
C2	0	1	1	0	0	1	1	0	0	1	1	0	**0**

Calculation of C2:

C2 = D0 ⊕ D2 ⊕ D3 ⊕ D5 ⊕ D6 =

C2 = 1 ⊕ 0 ⊕ 1 ⊕ 1 ⊕ 0 = 1

Hamming Code

	D7	D6	D5	D4	C8	D3	D2	D1	C4	D0	C2	C1	
	12	11	10	9	8	7	6	5	4	3	2	1	
	0	0	1	1	C8	1	0	0	C4	1	C2	C1	**C0**
C4	1	0	0	0	0	1	1	1	1	0	0	0	**0**

Calculation of C4:

C4 = D1 ⊕ D2 ⊕ D3 ⊕ D7 =

C4= 0 ⊕ 0 ⊕ 1 ⊕ 0 = 1

Hamming Code

	D7	D6	D5	D4	C8	D3	D2	D1	C4	D0	C2	C1	
	12	11	10	9	8	7	6	5	4	3	2	1	
	0	0	1	1	C8	1	0	0	C4	1	C2	C1	**C0**
C8	1	1	1	1	1	0	0	0	0	0	0	0	**0**

Calculation of C8:

C8 = D4 ⊕ D5 ⊕ D6 ⊕ D7 =

C8= 1 ⊕ 1 ⊕ 0 ⊕ 0 = 0

Hamming Code

C1 = 1, C2 = 1 , C4 = 1, and C8 = 0

0111

	D7	D6	D5	D4	C8	D3	D2	D1	C4	D0	C2	C1	
	12	11	10	9	8	7	6	5	4	3	2	1	
	0	0	1	1	C8	1	0	0	C4	1	C2	C1	**C0**
	0	**0**	**1**	**1**	**0**	**1**	**0**	**0**	**1**	**1**	**1**	**1**	**0**

Hamming Code

We Assumed the received data has an error at bit 6 → D2:

	D7	D6	D5	D4	C8	D3	D2	D1	C4	D0	C2	C1		
	12	11	10	9	8	7	6	5	4	3	2	1		
	0	0	1	1	C8	1	0	0	C4	1	C2	C1	**C0**	
	0	**0**	**1**	**1**	**0**	**1**	**1**	**0**	**1**	**1**	**1**	**1**	**0**	
C1	0	1	0	1	0	1	0	1	0	1	0	1	**0**	**1**
C2	0	1	1	0	0	1	1	0	0	1	1	0	**0**	**0**
C4	1	0	0	0	0	1	1	1	1	0	0	0	**0**	**0**
C8	1	1	1	1	1	0	0	0	0	0	0	0	**0**	**0**

C1= 1 ⊕ 0 ⊕ 1 ⊕ 1 ⊕ 0= 1

C2= 1 ⊕ 1 ⊕ 1⊕ 1 ⊕ 0 = 0

C4= 0 ⊕ 1 ⊕ 1 ⊕ 0 = 0

C8= 1 ⊕ 1 ⊕ 0 ⊕ 0 = 0

Hamming Code:

We need to know where is the error:
By XORing the two C1, C2, C3, and C4 at TX and RX, we will find the number of bit.

$0111 \oplus 0001 = 0110$
0110 = 6 its mean the bit should be flipped

	D7	D6	D5	D4	C8	D3	D2	D1	C4	D0	C2	C1		
	12	11	10	9	8	7	6	5	4	3	2	1		
	0	0	1	1	C8	1	0	0	C4	1	C2	C1	**C0**	
	0	**0**	**1**	**1**	**0**	**1**	**1**	**0**	**1**	**1**	**1**	**1**	**0**	

Correction:

```
Corrected_Bit[D] = Received_Bit[D] XOR 1
```

$1 \oplus 1 = 0$

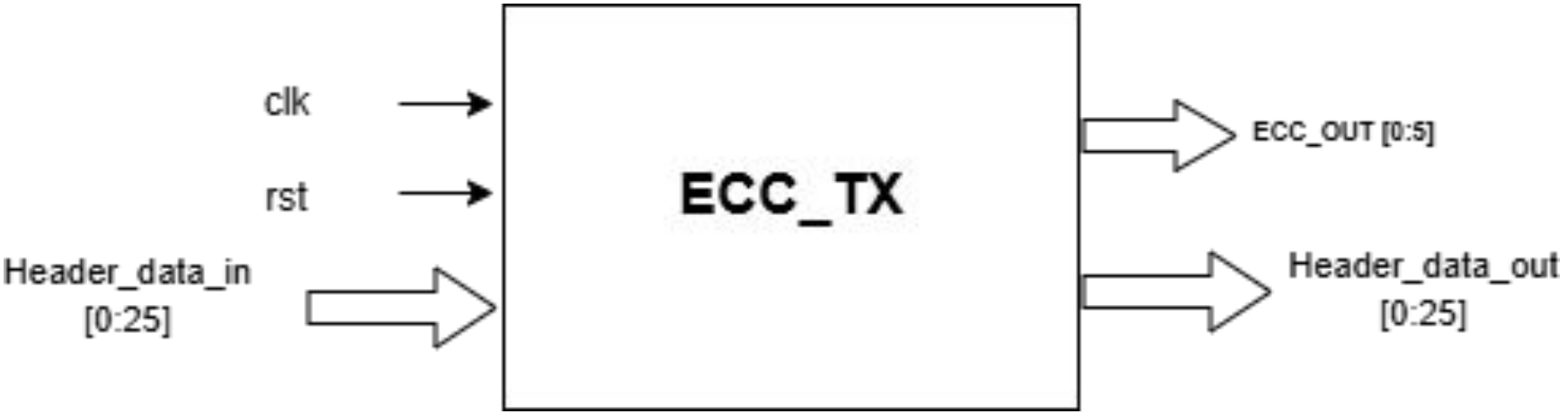
ECC Parity Generation Rules

Bit	P7	P6	P5	P4	P3	P2	P1	P0	Hex
0	0	0	0	0	0	1	1	1	0x07
1	0	0	0	0	1	0	1	1	0x0B
2	0	0	0	0	1	1	0	1	0x0D
3	0	0	0	0	1	1	1	0	0x0E
4	0	0	0	1	0	0	1	1	0x13
5	0	0	0	1	0	1	0	1	0x15
6	0	0	0	1	0	1	1	0	0x16
7	0	0	0	1	1	0	0	1	0x19
8	0	0	0	1	1	0	1	0	0x1A
9	0	0	0	1	1	1	0	0	0x1C
10	0	0	1	0	0	0	1	1	0x23
11	0	0	1	0	0	1	0	1	0x25
12	0	0	1	0	0	1	1	0	0x26
13	0	0	1	0	1	0	0	1	0x29
14	0	0	1	0	1	0	1	0	0x2A
15	0	0	1	0	1	1	0	0	0x2C
16	0	0	1	1	0	0	0	1	0x31
17	0	0	1	1	0	0	1	0	0x32
18	0	0	1	1	0	1	0	0	0x34
19	0	0	1	1	1	0	0	0	0x38
20	0	0	0	1	1	1	1	1	0x1F
21	0	0	1	0	1	1	1	1	0x2F
22	0	0	1	1	0	1	1	1	0x37
23	0	0	1	1	1	0	1	1	0x3B
24	0	0	1	1	1	1	0	1	0x3D
25	0	0	1	1	1	1	1	0	0x3E
26	0	1	0	0	0	1	1	0	0x46
27	0	1	0	0	1	0	0	1	0x49
28	0	1	0	0	1	0	1	0	0x4A
29	0	1	0	0	1	1	0	0	0x4C
30	0	1	0	1	0	0	0	1	0x51
31	0	1	0	1	0	0	1	0	0x52

ECC

At transmitter side:

ECC



ECC

At transmitter side:

Signal Name	Direction	Description / Function
clk	Input	System clock signal that synchronizes all operations in the ECC transmitter block.
rst	Input	Reset signal used to initialize or clear the ECC_TX module. When asserted, all registers and internal states are reset to default.
Header_data_in[0:25]	Input	Input data bus containing the header information to be encoded. This is the original unprotected data before error correction bits are added.
ECC_OUT[0:5]	Output	Generated ECC (Error Correction Code) bits. These are parity or redundant bits calculated from the input data to detect and correct transmission errors.
Header_data_out[0:25]	Output	The same header data forwarded to the next block or output interface, kept unchanged but synchronized with the generated ECC bits.

ECC

At receiver side:

ECC

ECC

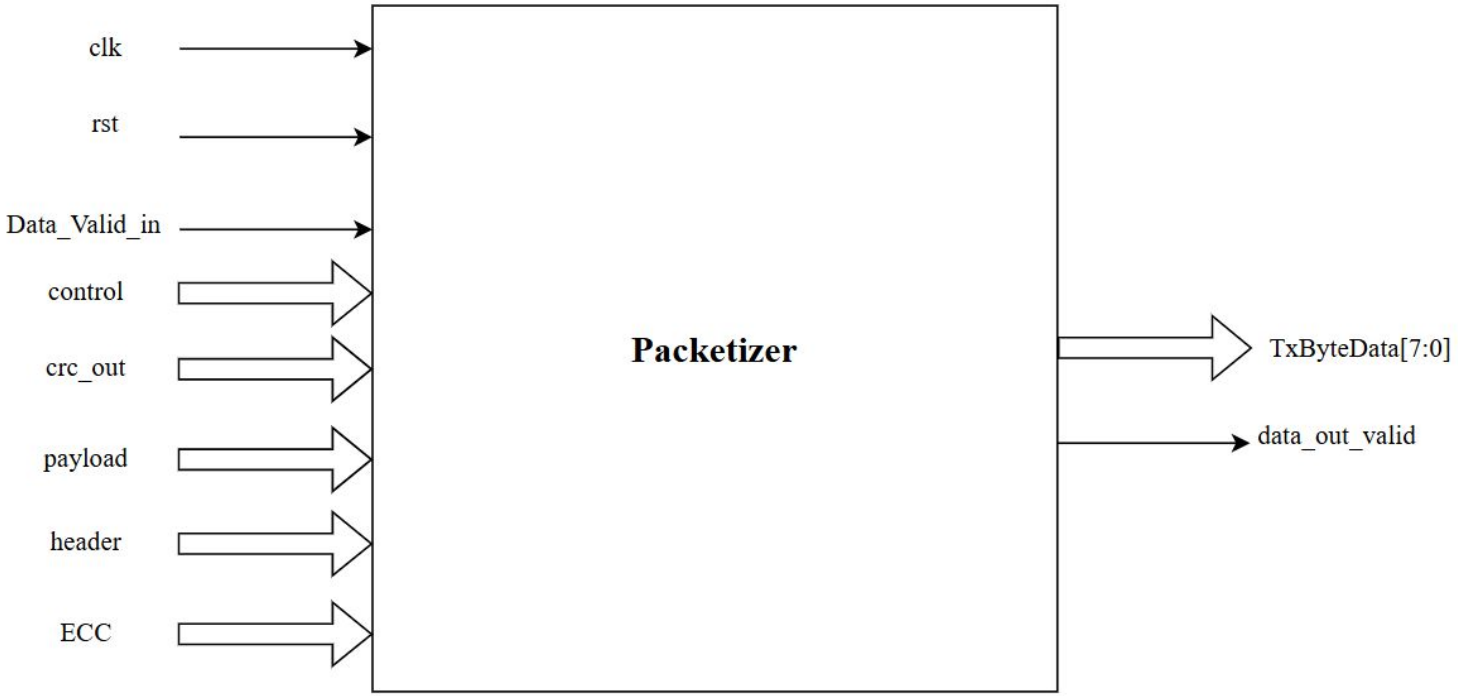
At receiver side:

Signal Name	Direction	Description / Function
clk	Input	System clock signal that synchronizes the receiver-side ECC decoding operations. All error detection and correction occur on clock edges.
rst	Input	Reset signal to initialize the ECC_RX module. When active, it clears all internal registers, flags, and outputs.
received_Header_data_in[0:31]	Input	Input data bus containing the **received 26-bit header** concatenated with the **6 ECC bits** generated by ECC_TX (26 + 6 = 32 bits). This is the potentially corrupted data received from the communication channel.
single_bit_error_flag	Output	Set to logic '1' if a **single-bit error** is detected and corrected. Indicates successful correction without retransmission.
double_bit_error_flag	Output	Set to logic '1' if a **double-bit error** is detected. Indicates that the error cannot be corrected and the data is invalid.
corrected_header_data_out[0:25]	Output	Output data bus containing the **corrected header** after ECC decoding. If no errors are detected or if a single-bit error is corrected, this matches the original header sent by ECC_TX.

SI-VISION ACADEMY

Packetizer

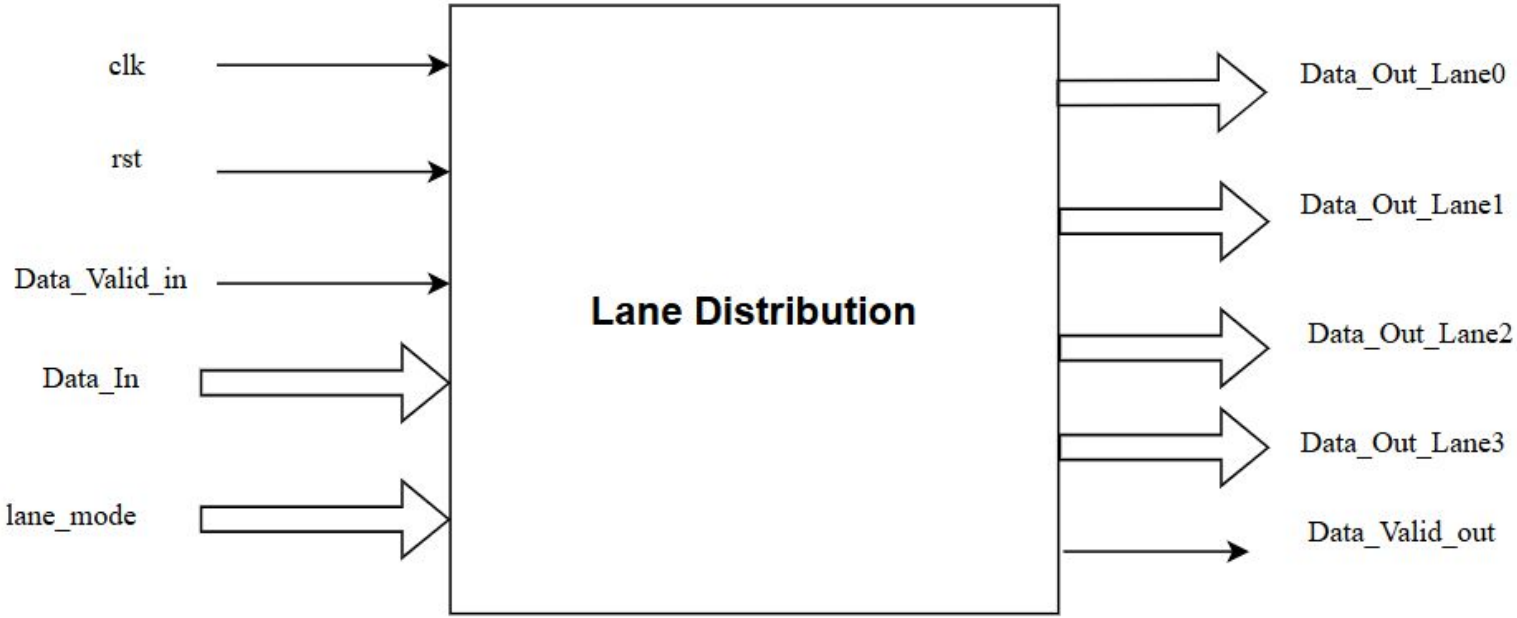
Packetizer


clk
rst
Data_Valid_in
control
crc_out
payload
header
ECC
Packetizer
TxByteData[7:0]
data_out_valid

SI-VISION ACADEMY

Lane Distributor

Lane Distributor



Lane Distributor

Signal Name	Direction	Description
clk	Input	Clock signal
rst	Input	Asynchronous reset signal
Data_Valid_in	Input	A control signal asserted by the packetizer block to indicate that the data presented on Data_In is valid and ready to be transmitted.
Data_In	Input	The byte-stream of CSI-2 packets (headers, payload, footers) arriving from the packetizer/low-level protocol layer, typically. The Lane Distributor serializes this stream onto the output lanes.
lane_mode	Input	A configuration input to specify the number of active lanes (N) to distribute data across lanes.

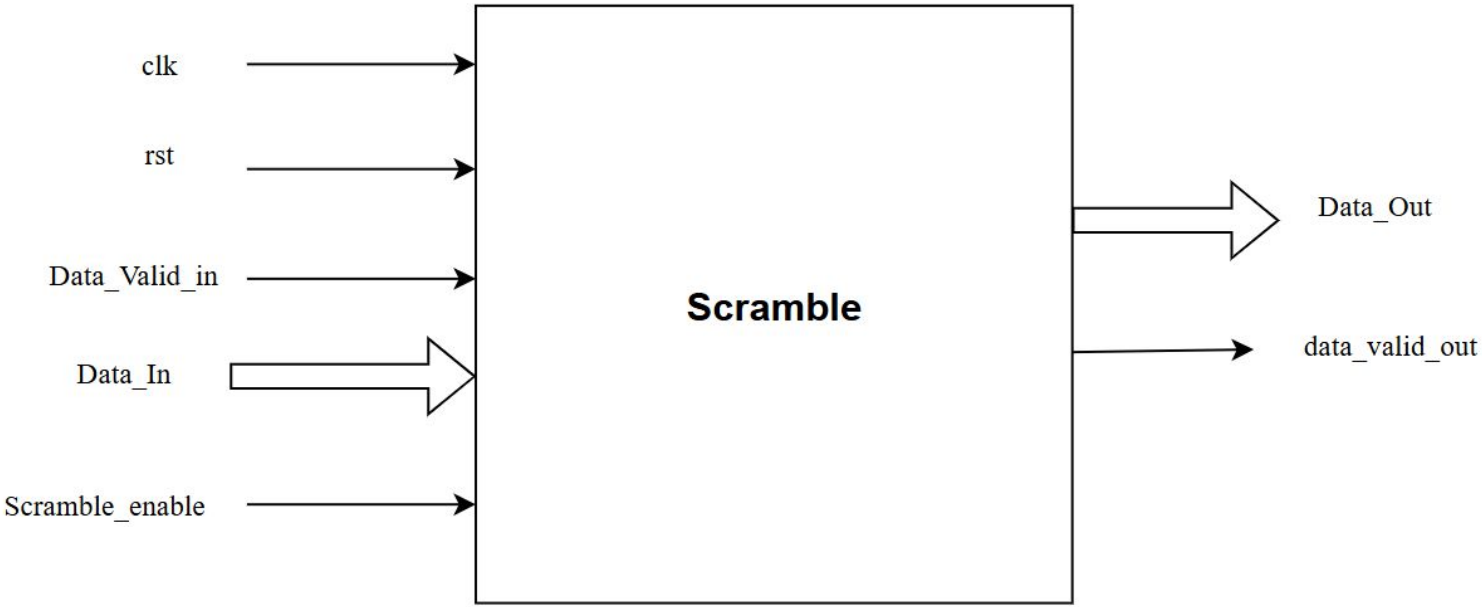
Lane Distributor

Signal Name	Direction	Description
Data_Out_Lane0 Data_Out_Lane1 Data_Out_Lane2 Data_Out_Lane3	Output	The distributed data that is now split across multiple lanes for parallel transmission.
Data_Valid_out	Output	Confirms that the data distributed to the lanes is valid and ready for further processing. When this signal is active, the downstream block can process the parallel data. It ensures the data integrity.

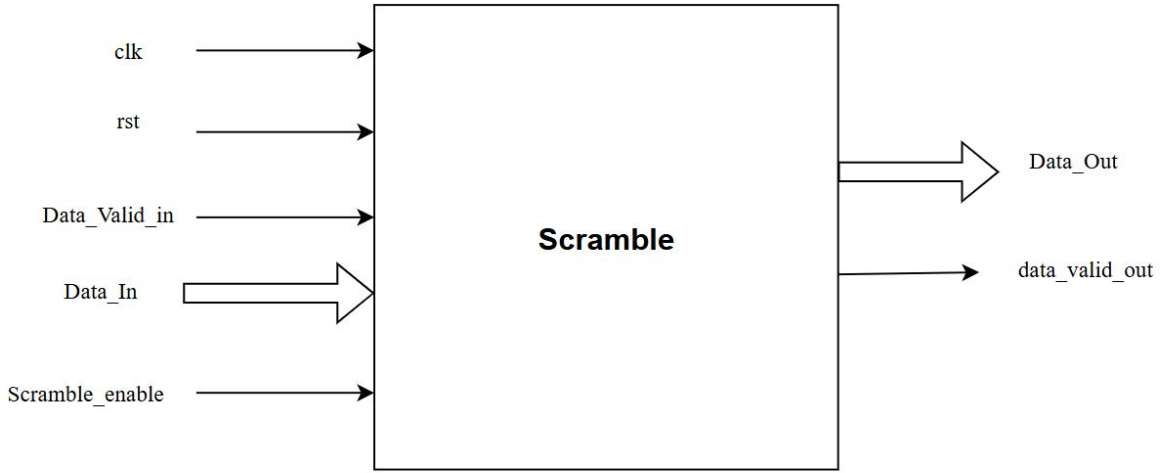
SI-VISION ACADEMY

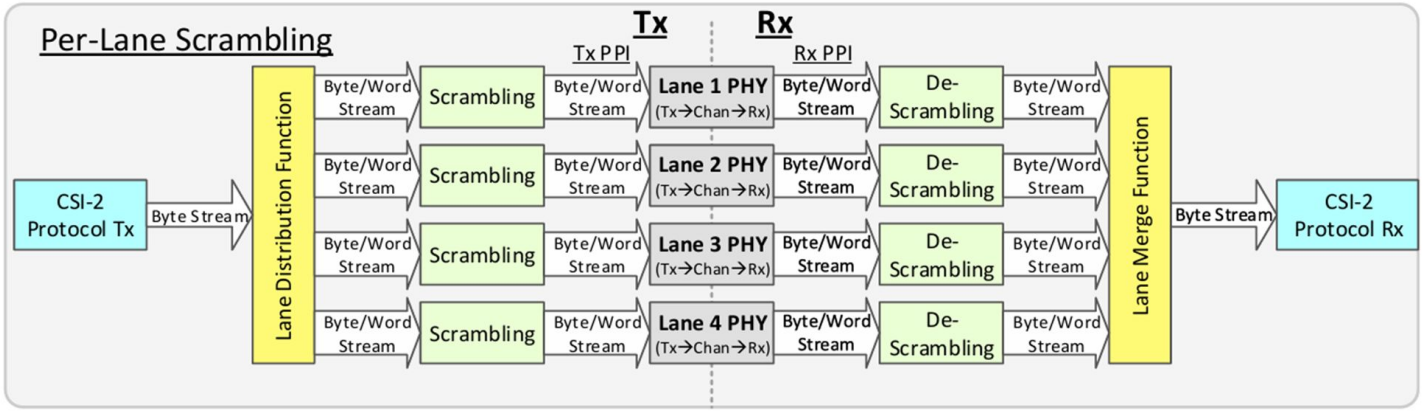
Scramble

Scramble


clk
rst
Data_Valid_in
Data_In
Scramble_enable
Scramble
Data_Out
data_valid_out

Scramble


clk
rst
Data_Valid_in
Data_In
Scramble_enable
Scramble
Data_Out
data_valid_out


Per-Lane Scrambling
Tx
Rx
Tx PPI
Rx PPI
CSI-2 Protocol Tx
Byte Stream
Lane Distribution Function
Byte/Word Stream
Scrambling
Lane 1 PHY (Tx→Chan→Rx)
Lane 2 PHY (Tx→Chan→Rx)
Lane 3 PHY (Tx→Chan→Rx)
Lane 4 PHY (Tx→Chan→Rx)
De-Scrambling
Lane Merge Function
Byte Stream
CSI-2 Protocol Rx

Scramble

Signal Name	Direction	Description
clk	Input	Clock signal
rst	Input	Asynchronous reset signal
Data_Valid_in	Input	A control signal indicating valid incoming data (from the Lane Distributor).
Data_In	Input	Data stream for a specific lane from the Lane Distributor. This includes unscrambled packet headers and scrambled packet payloads
Scramble_enable	Input	A control signal that enables or disables the scrambling logic. Scrambling is typically applied to **long packet payload data only**

Scramble

Signal Name	Direction	Description
Data_Out	Output	The output data stream (scrambled or unscrambled, depending on the data type and Scramble_enable). This output connects to the D-PHY serializer via the PPI.
data_valid_out	Output	A control signal indicating valid output data, typically a buffered version of Data_Valid_in.

SI-VISION ACADEMY

Receiver

CSI-2 Receiver

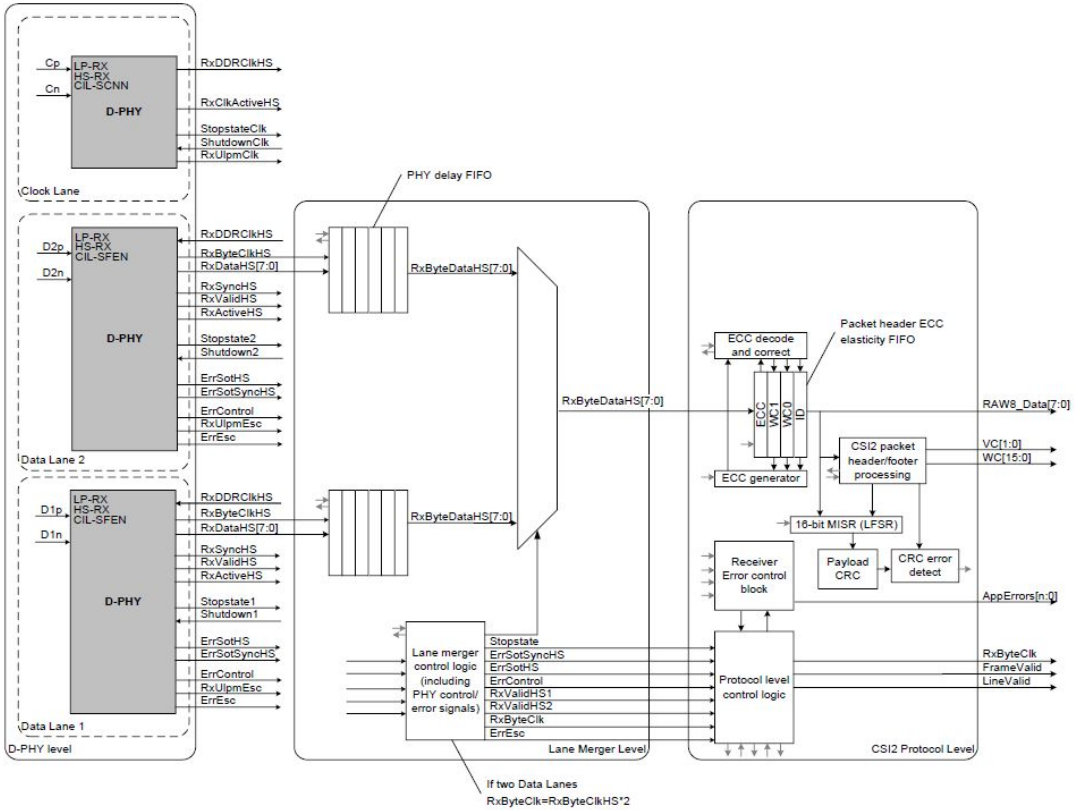
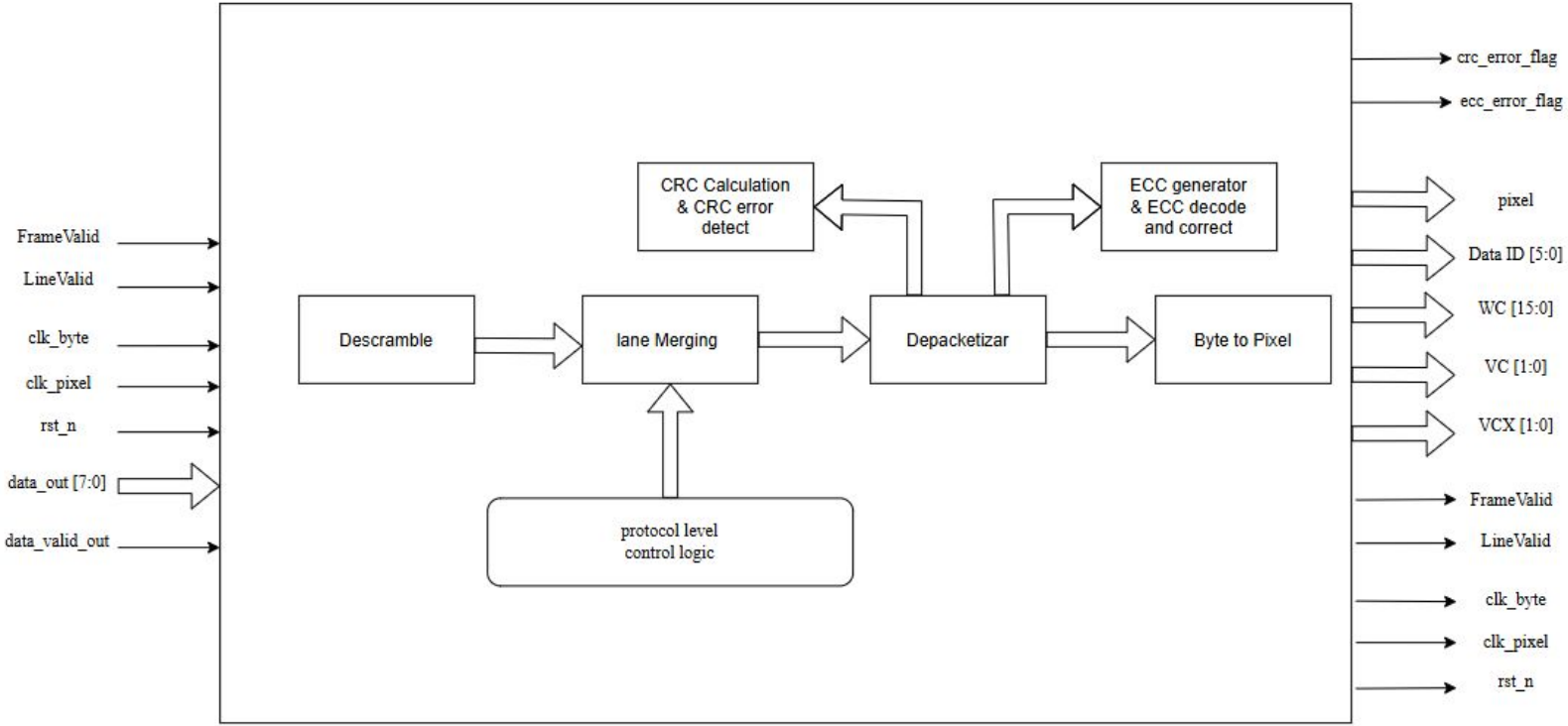


Figure 214 CSI-2 Receiver Block Diagram

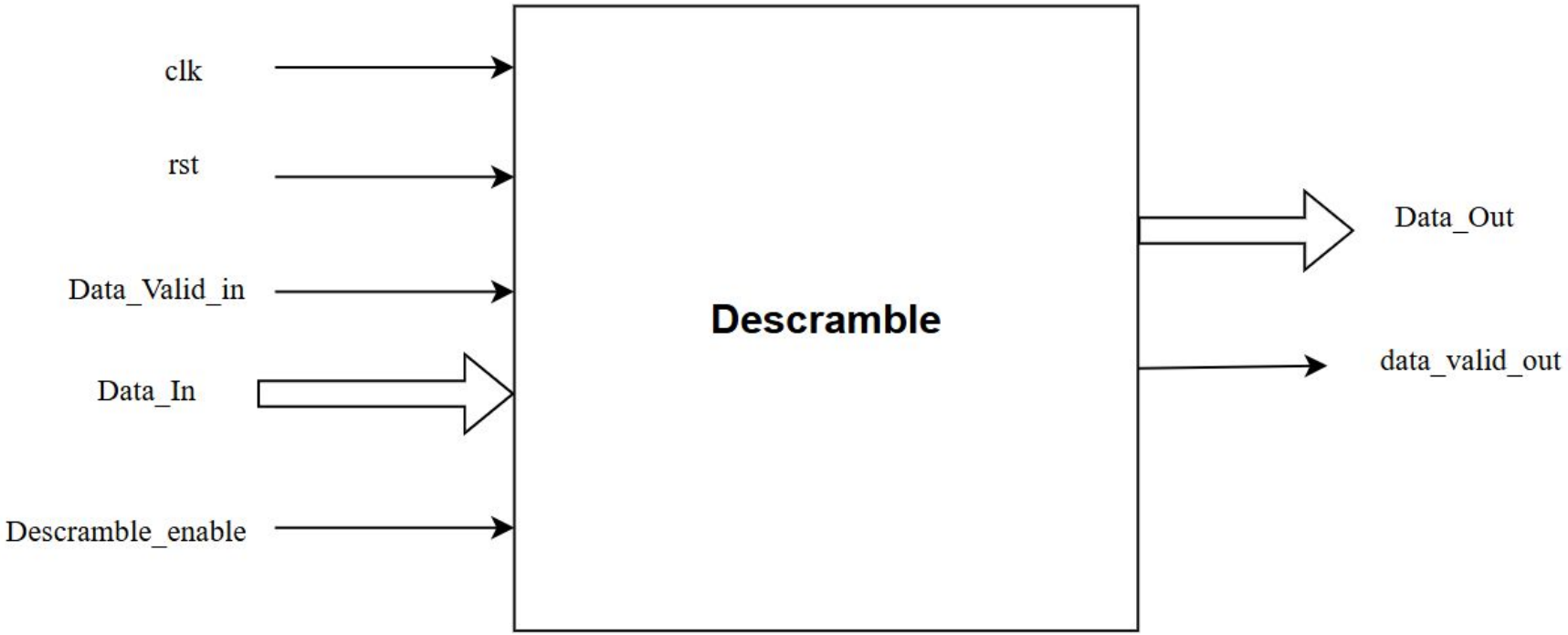
CSI-2 Receiver



SI-VISION ACADEMY

Descramble

Descramble


clk
rst
Data_Valid_in
Data_In
Descramble_enable
Descramble
Data_Out
data_valid_out

Descramble

Signal Name	Direction	Description
clk	Input	Clock signal
rst	Input	Asynchronous reset signal
Data_Valid_in	Input	A control signal indicating valid incoming data from the D-PHY PPI.
Data_In	Input	The scrambled or unscrambled byte stream for a specific lane received from the D-PHY PPI
Descramble_enable	Input	A control signal that enables or disables the descrambling logic, applied conditionally to the long packet payload data.

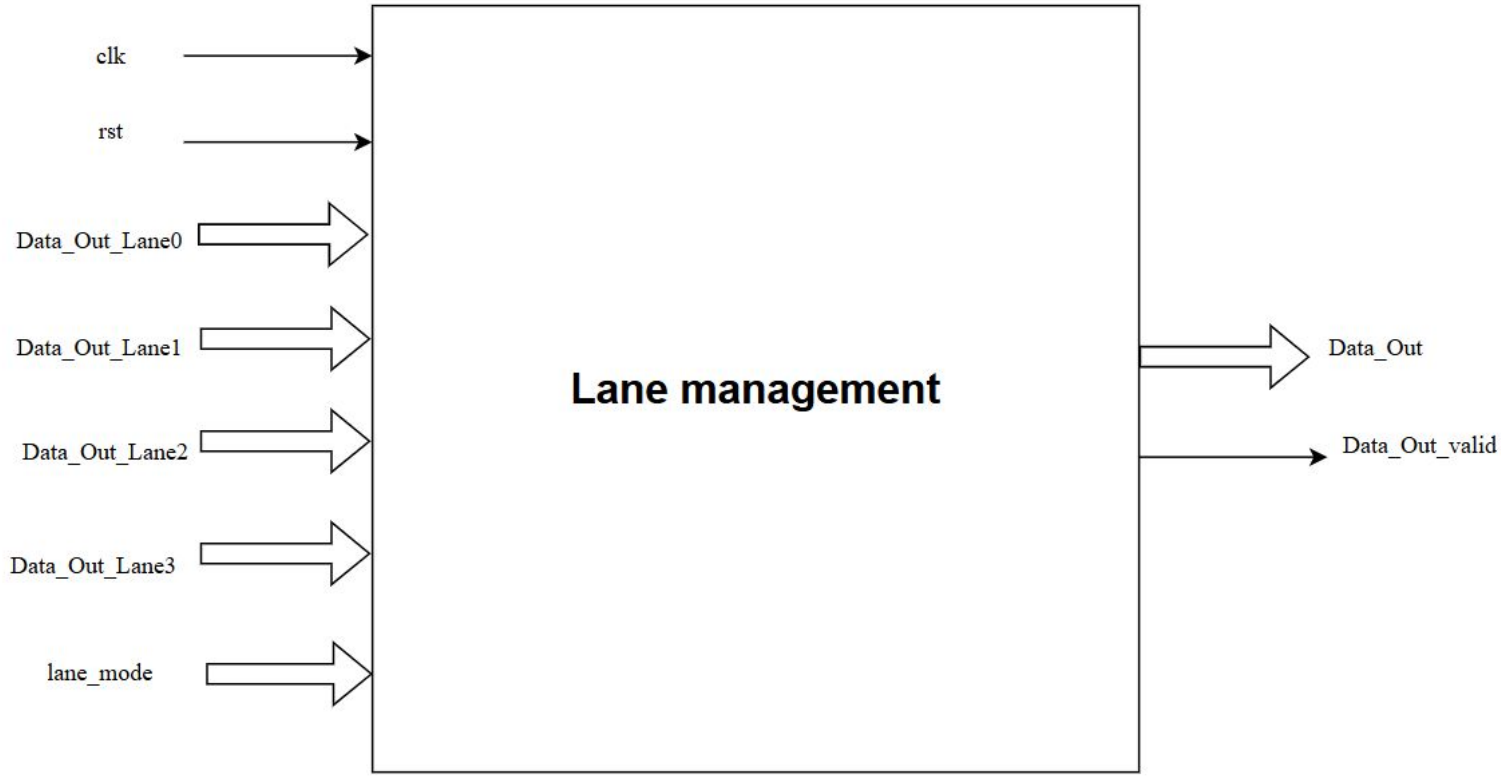
Descramble

Signal Name	Direction	Description
Data_Out	Output	The recovered data stream (unscrambled data). This output feeds the Lane Management block.
data_valid_out	Output	A control signal indicating valid output data.

SI-VISION ACADEMY

Lane Management

Lane Management



Lane Management

Signal Name	Direction	Description
clk	Input	Clock signal
rst	Input	Asynchronous reset signal
Data_Out_Lane0 Data_Out_Lane1 Data_Out_Lane1 Data_Out_Lane1	Input	The unscrambled byte-stream input from each active lane (from the Descrambler blocks). The Lane Management block must align the timing of these streams before merging.
lane_mode	Input	Configuration input, identical to the transmitter side, specifying the number of active lanes (N) to manage and merge data from.

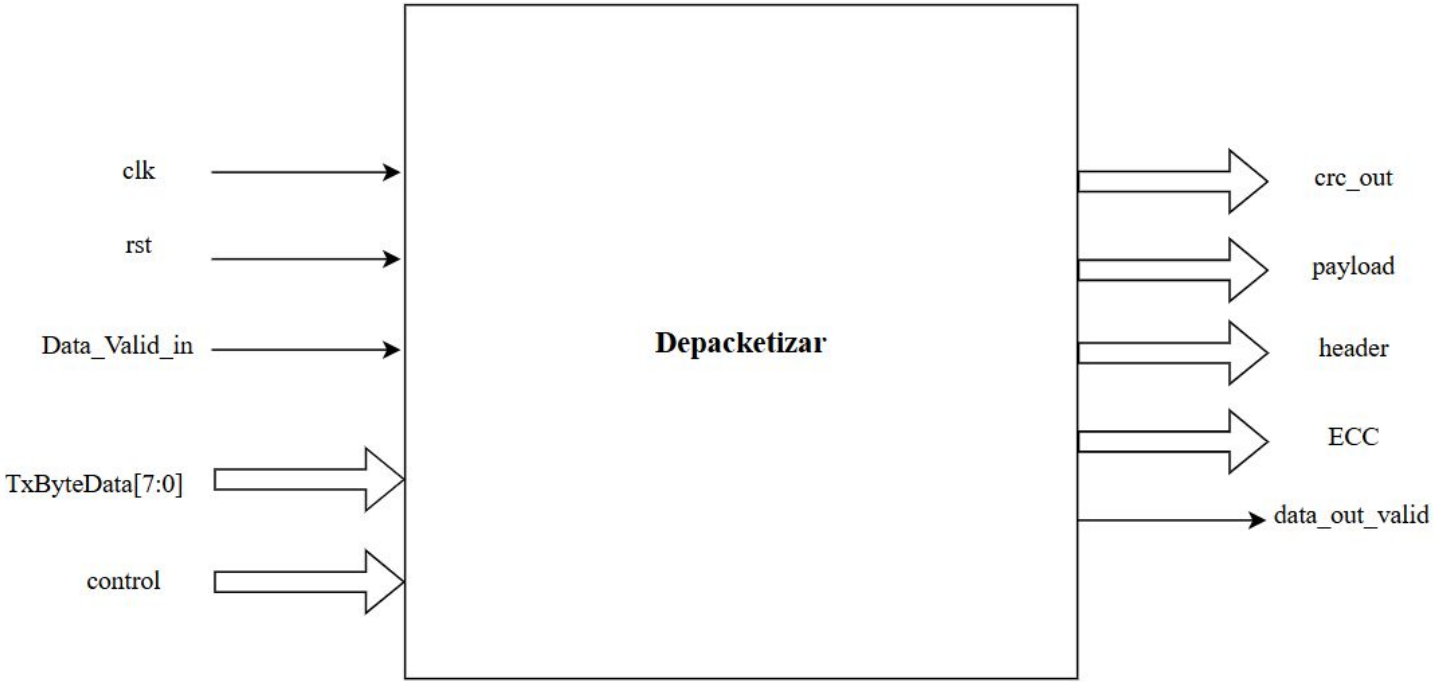
Lane Management

Signal Name	Direction	Description
Data_Out	Output	The final reconstructed, single byte-stream of CSI-2 packets. This stream maintains the original packet order (Byte 0, Byte 1, Byte 2, ...) and is passed to the next block (Depacketizer/ECC/CRC)
Data_Out_valid	Output	A control signal indicating that data is valid on the Data_Out bus, synchronized with the output data stream.

SI-VISION ACADEMY

Depacketizer

Depacketizer

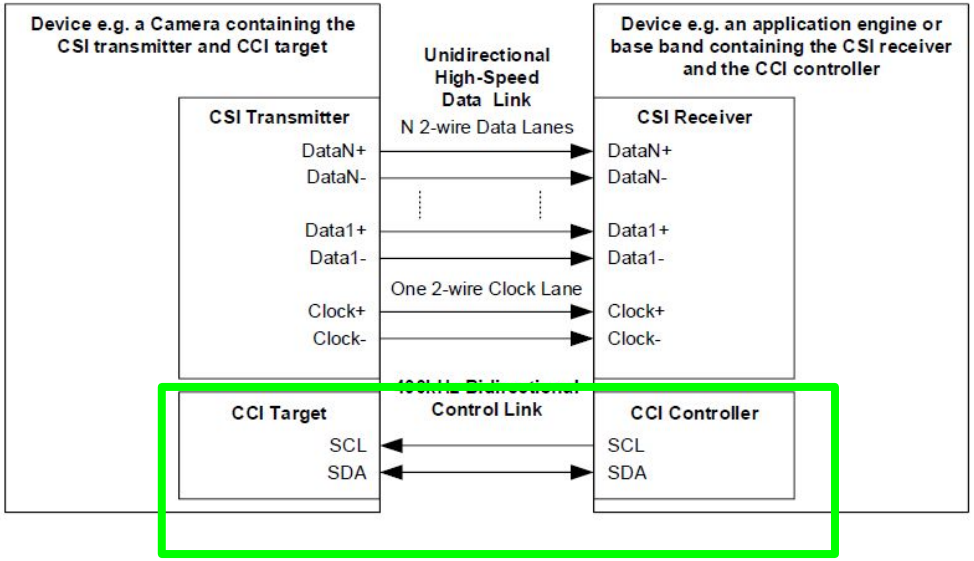


SI-VISION ACADEMY

Camera Control Interface

CCI-Camera Control Interface

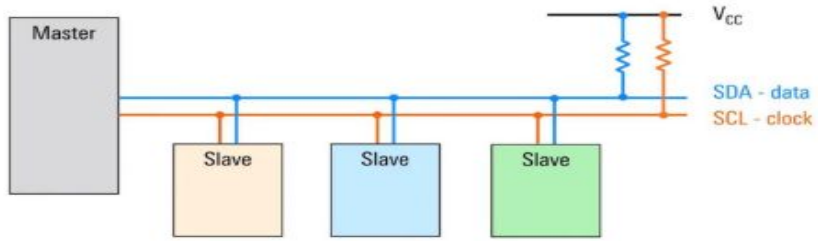
- The Camera Control Interface (CCI) is an I²C-based control bus used by the host processor to configure and control MIPI-compatible image sensors.
- It provides access to internal sensor registers (CCI Registers) that define operational parameters such as exposure, gain, and mode control.



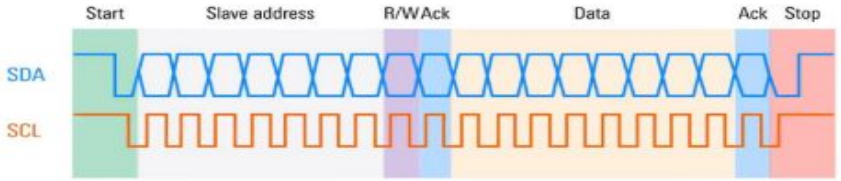
CCI-Camera Control Interface

- Inter-Integrated Circuit (I^2C or I2C)
- Developed in 1982 by Philips (now NXP)
- Very common
- Primarily used for short distance data communications
- Synchronous master-slave protocol
- Both master and slaves can send/receive data
- Bidirectional, half duplex
- Can run at different speeds
- Two wires: serial clock (SCL) and serial data (SDA)

Basic I2C Topology



I2C Frames



- ACK stands for "acknowledge." It is a signal used by the receiving device to indicate to the sending device that it has successfully received a byte of data.

CCI-Camera Control Interface-System Architecture

Block	Function
Host Controller (Master)	Initiates all read/write transactions over CCI. Generates SCL and drives SDA.
Image Sensor (Slave)	Responds to Host CCI transactions; exposes registers mapped through a defined address space.
CCI Bus	Bi-directional I²C-style 2-wire interface connecting Host and sensors.
Internal Register Bank	Organized register map (0x0000–0x2FFF) accessed via CCI. Contains CCS registers for configuration, timing, and capability reporting.
Clock and Power Control	EXTCLK, XSHUTDOWN, and supply lines synchronize power-up, standby, and streaming transitions.

CCI-Camera Control Interface-Signals

Signal	Direction	Description
SCL	Host → Sensor	Serial clock line. Typically 100–400 kHz for standard mode.
SDA	Bidirectional	Serial data line for register read/write operations.
XSHUTDOWN	Host → Sensor	Hardware standby control signal (low = reset/off).
EXTCLK	Host → Sensor	External reference clock for sensor timing and PLL.

CCI-Camera Control Interface-Buse characteristics

Parameter	Typical Value
Bus Type	I²C compatible
Data Width	8-bit register access; 16-bit register index (address)
Register Indexing	0x0000–0x2FFF for CCS-defined registers
Clock Frequency	Standard (≤400 kHz) or Fast Mode (≤1 MHz), implementation-dependent

CCI-Camera Control Interface-Register Map Organization I

Range	Access	Category
0x0000–0x00FF	RO	Status Registers
0x0100–0x01FF	RW	Operating Mode / Setup
0x0200–0x02FF	RW	Integration Time & Gain
0x0300–0x03FF	RW	Video Timing
0x0400–0x04FF	RW	Image Scaling
0x0500–0x05FF	RW	Compression
0x0800–0x08FF	RW	CSI-2/PHY Configuration

CCI-Camera Control Interface-Register Map Organization II

Range	Access	Category
0x0900–0x09FF	RW	Binning
0x0A00–0x0AFF	RW	Data Transfer Interface
0x0B00–0x0BFF	RW	Image Processing
0x1000–0x1FFF	RO	Parameter Limits
0x2000–0x2FFF	RW	Image Statistics
0x3000–0xFFFF	Vendor-specific Extensions	

CCI-Camera Control Interface-Example Functional Registers

Register	Width	Function
output_format_ctrl	8 bit	CSI-2 data format selection
mode_select	8 bit	Switch between standby and streaming modes.
extclk_frequency_mhz	16 bit	Input clock frequency reference.
vt_pix_clk_div, pll_multiplier	16 bit	Clock tree configuration (PLL, dividers).

CCI-Camera Control Interface-operating states

Mode	Description	CCI Activity
Power-Off	No supply; all interfaces off.	None
HW Standby	Power on, XSHUTDOWN low.	No CCI
SW Standby	Power on, XSHUTDOWN high.	CCI active, configuration mode
Streaming	Image data output via CSI-2.	CCI active; NACKs if busy

SI-VISION ACADEMY

Timeline

Scope

CSI-2 GP Timeline				
Item	Start Date	Due Date	Weeks	Note
Architecture	25/09/2025	31/10/2025	5	
RTL + Block-level testbenches	1/11/2025	1/1/2026	8.5	Midterm Exams from 7/11 to 15/11
Spyglass Synthesis + Formal verification	20/1/2026	20/2/2026	4	Finals Exams from 3/1 to 19/1
UVM	7/1/2026	7/3/2026	8	
DFT	8/3/2026	22/03/2026	2	
ASIC (backend) flow	22/03/2026	16/05/2026	7	Midterm Exams from 28/3 to 4/4

Scope

CSI-2 GP Timeline

Item	Start Date	Due Date	Weeks
Architecture	25/09/2025	31/10/2025	5
RTL + Block-level testbenches	01/11/2025	01/01/2026	8.5
Spyglass	01/01/2026	01/02/2026	4
Synthesis + Formal verification			
UVM		01/03/2026	8
DFT	01/03/2026	15/03/2026	2
ASIC (backend) flow	16/03/2026	16/05/2026	8.5
FPGA flow - SoC level (optional)	17/05/2026	-	

SI-VISION ACADEMY

References

References

[1] MIPI Alliance, "Specification for Camera Serial Interface 2 (CSI-2®) Version 4.0.1," MIPI Alliance, May 23, 2022

[2] MIPI Alliance, "Specification for D-PHY Version 2.5," MIPI Alliance, Jul. 5, 2019. [Online].

[3] Gowin MIPI Byte to Pixel Converter IP |Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2753/document/6820dee2cd163.pdf

[4] Intel Corporation, *"MIPI CSI-2 Intel® FPGA IP User Guide,"* Doc. No. 813926, Version 1.1.0, Apr. 26, 2024. Online. Available: https://www.intel.com/content/www/us/en/docs/programmable/813926.html

[5] Gowin MIPI Pixel to Byte Converter IP |Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2752/document/6820df0ae4095.pdf

[6] Lattice Semiconductor, "Cyclic Redundancy Check Reference Design," *Lattice Reference Design*, accessed Oct. 18, 2025. [Online]. Available: https://www.latticesemi.com/products/designsoftwareandip/intellectualproperty/referencedesigns/referencedesigns01/cyclicredundancycheck

[7] *Mipi Camera Command Set (MIPI CCS)*. MIPI. (n.d.). https://www.mipi.org/specifications/camera-command-set

SI-VISION ACADEMY

Thank You!